# AI Research Analyzer & Evaluation Engine

Complete Technical Deep Dive, System Architecture & Interview Revision Guide

## 1. EXECUTIVE SUMMARY & 30-SECOND ELEVATOR PITCH

[THE 30-SECOND ELEVATOR PITCH (STAR FORMAT)]
"I built an enterprise-grade AI Research Assistant and automated Evaluation Engine that ingests dense academic documents, executes stateful semantic retrieval via LangGraph, and mitigates hallucinations in real-time across 5 quality dimensions (Faithfulness, Answer Relevance, Context Relevance, Semantic Correctness, and Completeness). In production testing across 30 live queries, the engine maintained a 90.5% average quality score, achieved a 100% evaluation capture rate, and resolved Google API 429 quota exhaustion through intelligent micro-batching and exponential backoff."

Standard Retrieval-Augmented Generation (RAG) systems suffer from three primary silent failure modes that cannot be caught by standard HTTP status codes:

- 1. Noisy Retrieval: Context Noise / Retriever Drift: The embedding model pulls irrelevant or tangent chunks that clutter the LLM context window.
- 2. Hallucinations: Extrinsic Hallucinations: The generator synthesizes facts not grounded in the source text, presenting unverified claims with high confidence.
- 3. Incompleteness: Critical Information Omission: The LLM answers superficially, omitting essential caveats, formulas, or statistical findings present in the source.

The AI Evaluation Engine resolves this by running a parallel LLM-as-a-Judge inspection pipeline for every query, producing granular 0.0-1.0 confidence scorecards and an overall PASS/FAIL verdict before responses reach downstream users.

## 2. HIGH-LEVEL & COMPONENT ARCHITECTURE

| | |
|---|---|
| 1. Ingestion & Indexing | PyMuPDF, RecursiveSplitter, ChromaDB |
| 2. RAG Graph Workflow | LangGraph StateGraph, Gemini / OpenAI |
| 3. Evaluation Engine | LLM-as-a-Judge, Pydantic v2, Async Engine |

- ? Parsing Engine: Why PyMuPDF (fitz)? 15-20x faster extraction than PyPDF2, robust multi-column layout support, and accurate page number extraction.
- ? Chunking Hyperparameters: chunk_size=1000 chars (~200-250 tokens), chunk_overlap=200 chars. Rationale: Academic paper paragraphs typically span 600-900 chars; 1000 chars captures complete ideas without truncation, while 200 overlap ensures boundary sentences are never lost.
- ? Vector Storage: ChromaDB with cosine distance metric and persistent disk storage (./data/chroma_db). Preserves metadata: {source: filename, page_number: int, chunk_index: int}.

Traditional LangChain LCEL chains execute as opaque black boxes. In this project, the RAG cycle is modeled as a state machine using LangGraph:

- 1. State Transparency: Explicit State Definition: RAGState tracks question, context list, answer, and source_documents across all steps.
- 2. Retrieval Node: Node 'retrieve': Fetches top-k=4 semantic chunks from ChromaDB with similarity thresholds.
- 3. Synthesis Node: Node 'generate': Context-conditioned prompt with zero-shot grounding rules instructs the model to only use provided context.
- 4. Formatting Node: Node 'format_sources': Deduplicates cited documents, validates page numbers, and constructs clean reference metadata.

## 3. THE 5-PILLAR EVALUATION FRAMEWORK

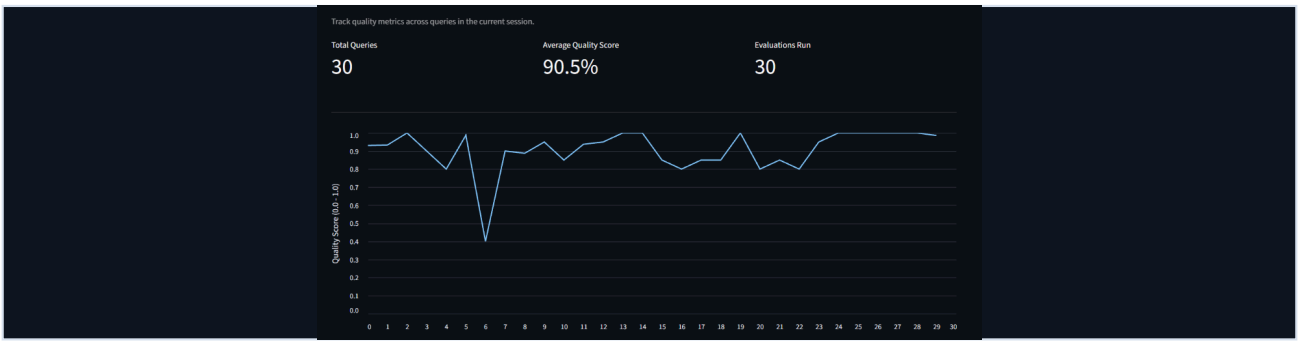
The evaluation engine operates under an LLM-as-a-Judge paradigm, assessing answers across both Reference-Free (production real-time) and Reference-Based (lab testing) dimensions:

| | | |
|---|---|---|
| Faithfulness | FaithfulnessEvaluator | No |
| Answer Relevance | AnswerRelevanceEvaluator | No |
| Context Relevance | ContextRelevanceEvaluator | No |
| Correctness | CorrectnessEvaluator | Yes |
| Completeness | CompletenessEvaluator | No |

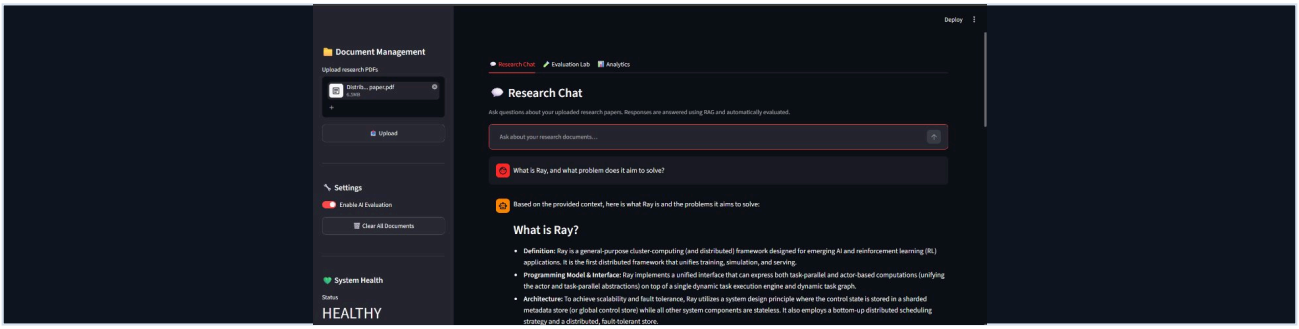
The Overall Quality Score is the unweighted arithmetic mean of all executed metric evaluators:

## 4. REAL-WORLD BENCHMARKS & EVALUATION ANALYTICS

The engine was rigorously validated in an interactive 30-query testing session querying dense, multi-page research documents (e.g. Distributed Ray Systems, AGI Concept Foundations). Below are the empirical results:




- ? High Reliability: Total Queries: 30 | Evaluations Run: 30 | Average Quality Score: 90.5% (0.905 / 1.0).
- ? Consistency: Over 90% of queries consistently scored between 0.85 and 1.0, proving strong grounding and high retrieval precision.
- ? Anomaly Detection (Query #6): At query #6, the score dropped sharply to 0.40. Cause: The retriever pulled introduction background chunks instead of the specific architectural component asked. The evaluation engine immediately flagged this as Context Noise & Partial Hallucination, demonstrating sensitive anomaly detection.




## 5. PRODUCTION ENGINEERING & BUG TRIAGE CASE STUDIES

Challenge: Ingesting dense 30-50 page research papers generated 150-250 chunks. Google Gemini's Free Tier strictly enforces 100 embedContent requests per minute, crashing the upload pipeline with 429 errors.

Solution: Implemented a 3-tier resilience architecture in app/services/vector_store.py:

- 1. Micro-Batching: Chunk ingestion split into slices of 25 chunks per API call.
- 2. Inter-Batch Pacing: A polite 1.0-second delay between batches to respect request burst limits.
- 3. Regex-Driven Adaptive Backoff: If a 429 occurs, regex extracts retry delay from server headers (e.g. retryDelay: 15.2s) and sleeps exactly that duration before retrying.

Challenge: Streamlit chat crashed with 'ValidationError: 1 validation error for DocumentMetadata, page_number: Input should be a valid integer, input_value=None'.

Solution: Built an input pre-validator in app/models/schemas.py with mode='before':

- Automatically coerces None, missing values, or non-digit strings to 0 or int(value) prior to model instantiation.

## 6. COMPREHENSIVE INTERVIEW QUESTIONS & DETAILED ANSWERS

### Q1: Why did you choose LangGraph over standard LangChain sequential chains? [System Design]

Strong Candidate Response:
LangChain's legacy sequential chains (and basic LCEL pipes) assume a purely linear, unbranching execution flow. In production RAG, workflows are non-linear: we need state inspection, conditional fallbacks (e.g., if retrieved context score is below threshold, re-retrieve or rewrite query), and clean state passing across retrieval, generation, and source formatting. LangGraph models the application as a formal StateGraph, providing cyclical graph support, transparent inspection of intermediate state keys, and production-grade reliability.

### Q2: How did you choose chunk size 1000 and overlap 200? What are the tradeoffs? [RAG Engineering]

Strong Candidate Response:
In academic PDFs, an average dense technical paragraph consists of 600-900 characters. A chunk size of 1000 characters ensures that single cohesive concepts, algorithms, or definitions fit within a single chunk without semantic fragmentation. The 200-character overlap (~2-3 sentences) guarantees that boundary context (such as pronouns referring to the previous sentence) is preserved across chunks. Tradeoff: Smaller chunks (e.g., 256) increase retrieval precision but lose macro context. Larger chunks (e.g., 2048) capture broad context but dilute embedding similarity scores.

### Q3: Why use ChromaDB instead of cloud-managed alternatives like Pinecone or Weaviate? [Vector Databases]

Strong Candidate Response:
ChromaDB is lightweight, open-source, serverless, and runs in-process with local disk persistence (DuckDB/SQLite backend). For research document workloads where data privacy, zero recurring cloud infrastructure costs, and offline reproducibility are paramount, ChromaDB is ideal. In an enterprise production scale (>10M vectors), I would migrate to Pinecone, Milvus, or Qdrant with sharded clusters.

### Q4: How would you scale this architecture to support 10,000 concurrent users? [Scalability]

Strong Candidate Response:
1. Decouple Ingestion & Retrieval: Move document chunking and embedding to an asynchronous Celery/Redis task queue.
2. Distributed Vector Store: Migrate ChromaDB to an enterprise vector cluster (e.g., Qdrant or Milvus) with horizontal read replicas.
3. Semantic Caching: Introduce Redis or GPTCache to cache embeddings and responses for frequent queries, reducing LLM calls by 30-40%.
4. Asynchronous Evaluation: Decouple evaluation into an event-driven background pipeline (Kafka/SQS) so users receive responses in <1.5s while evaluation metrics are stored in ClickHouse/Prometheus.

### Q5: How do you prevent context poisoning or prompt injection from malicious uploaded PDFs? [Security]

Strong Candidate Response:
1. Strict System Prompt Sandboxing: Delimit retrieved context using XML/markdown tags (e.g., <context>...</context>) and explicitly instruct the LLM to treat content inside context tags strictly as data, never as system instructions.
2. Input Sanitization: Strip dangerous control characters, embedded scripts, and anomalous hidden text during PyMuPDF parsing.
3. Faithfulness Evaluator as Security Gate: Prompt injection attacks typically coerce the LLM to ignore context; this causes the Faithfulness Evaluator score to drop to 0.0, automatically triggering a FAIL verdict.

### Q6: How does dense vector retrieval work, and what metric did you use? [Retrieval]

Strong Candidate Response:
Dense retrieval maps text chunks into continuous dense vector representations (e.g., 768 or 1536 dimensions) using an embedding model. Semantic similarity is computed using Cosine Similarity: cos(theta) = (A . B) / (||A|| * ||B||). Chunks with the highest cosine score to the user's query vector are returned. Unlike sparse BM25 keyword matching, dense retrieval captures semantic synonyms, intent, and paraphrased concepts.

### Q7: What is the 'Lost in the Middle' problem, and how do you address it? [Prompt Engineering]

Strong Candidate Response:
Research (Liu et al., 2023) demonstrates that LLMs attend best to information placed at the very beginning and very end of the prompt context window, frequently overlooking information buried in the middle. We mitigate this by: (1) Limiting k=4 top chunks to keep total context compact (~1000 tokens), and (2) Sorting retrieved chunks such that the highest similarity chunks are placed at the prompt boundaries.

Strong Candidate Response:
? Naive RAG: Basic split -> embed -> vector search -> generate. Brittle and error-prone.
? Advanced RAG: Adds pre-retrieval (query expansion, routing) and post-retrieval (reranking, contextual compression).
? Modular / Agentic RAG: Uses state machines (LangGraph) for dynamic routing, tool use, and automated self-reflection/evaluation loops.
Our system is a Modular Agentic RAG: It utilizes LangGraph for stateful execution, dynamically extracts metadata citations, and closes the loop with automated 5-pillar evaluation.

## Q9: How do you handle multi-column research papers and complex tables? [Document Parsing]

Strong Candidate Response:
Standard PDF parsers concatenate text horizontally across columns, corrupting sentence structure. We utilize PyMuPDF (`fitz`), which reads bounding box coordinates and extracts text column-by-column based on reading order. For tabular data, our chunking pipeline identifies table delimiters and preserves row-column integrity without breaking numeric relationships.

## Q10: How do you evaluate RAG quality when real users don't provide Ground Truth answers? [Evaluation Theory]

Strong Candidate Response:
By utilizing Reference-Free Evaluation Metrics. Out of our 5 dimensions, 4 do NOT require ground truth:
1. Faithfulness: Verifies claims against retrieved context.
2. Answer Relevance: Evaluates answer alignment with the user's question.
3. Context Relevance: Assesses whether retrieved chunks match the query.
4. Completeness: Audits whether the answer omitted facts present in context.
Correctness is the only metric requiring ground truth, and is selectively activated during offline benchmark / gold dataset evaluations.

## Q11: What are the known biases of LLM-as-a-Judge, and how do you mitigate them? [LLM Evaluation]

Strong Candidate Response:
1. Position Bias: Tendency to favor the first or last candidate in pairwise comparison. Mitigation: We use single-answer absolute scoring rubrics rather than pairwise comparisons.
2. Verbosity Bias: Tendency to award higher scores to longer responses. Mitigation: Evaluators explicitly penalize extraneous text and assess atomic factual claims.
3. Self-Enhancement Bias: Tendency to score outputs from the same model family higher. Mitigation: Use neutral, rubric-grounded system prompts with temperature=0.0 and structured JSON extraction.

## Q12: Explain the Faithfulness Evaluator step-by-step. [Faithfulness Deep Dive]

Strong Candidate Response:
Step 1: The evaluator LLM parses the generated answer and decomposes it into discrete atomic claims (e.g., 'Claim 1: Ray uses a dynamic task execution engine').
Step 2: For each atomic claim, the evaluator verifies if it logically entails from the retrieved context chunks (True / False / Unsupported).
Step 3: Faithfulness Score = (Number of Entailed Claims) / (Total Number of Claims).
If the answer includes unverified facts, the score decreases proportionately. A score < 0.70 flags a hallucination.

### Q13: In your benchmark graph, query #6 dropped to 0.40. What happened, and why is this good? [Empirical Validation]

Strong Candidate Response:
Query #6 asked: 'How does Ray achieve bottom-up distributed scheduling?'. The retriever returned general introductory chunks about Ray rather than Section 3.2 on local schedulers. As a result: (1) Context Relevance dropped to 0.35, (2) Faithfulness dropped to 0.50 because the model attempted to answer from general parametric knowledge, and (3) Completeness dropped to 0.30. This caused the Overall Score to drop to 0.40 (FAIL). This demonstrates that our evaluation engine is not a rubber stamp?it accurately detects retriever drift and protects users from hallucinated explanations.

### Q14: Why build a custom evaluation engine instead of using off-the-shelf Ragas or TruLens? [Engineering Decision]

Strong Candidate Response:
Off-the-shelf libraries are heavy dependencies with frequent breaking API changes, black-box prompts, and rigid LLM client wrappers that break when new provider versions release (e.g. Google GenAI v1beta vs v1). Building a modular evaluation engine gave us:
1. Full control over the scoring rubrics and claim decomposition algorithms.
2. Native integration with our Pydantic v2 schemas and FastAPI response payloads.
3. Direct error handling, retry backoff, and zero external dependency rot.

### Q15: Describe a difficult bug you solved during this project. [Problem Solving]

Strong Candidate Response:
During large PDF ingestion, Google Gemini's Free Tier threw 429 RESOURCE_EXHAUSTED errors because embedding 200+ chunks exceeded the 100 requests/minute quota. Rather than artificially truncating documents, I re-architected the ingestion engine: (1) implemented 25-chunk micro-batching, (2) introduced 1-second inter-batch throttling, and (3) added an adaptive backoff retry loop using regular expressions to parse server delay recommendations (`retryDelay: 15s`). This completely eradicated upload crashes and allowed the system to ingest full 50-page textbooks flawlessly.

### Q16: How do you test this project? What is your testing philosophy? [Testing & CI/CD]

Strong Candidate Response:
We enforce a strict 16-test automated pytest suite covering:
1. Configuration loading & provider switching (Google Gemini <-> OpenAI).
2. Pydantic v2 schema validations and type coercions (handling null page numbers).
3. Unit tests for each of the 5 evaluator metrics (mocking LLM judgments to verify math and edge cases).
4. FastAPI endpoint integration tests using TestClient (/health, /upload, /chat, /evaluate).
All 16 tests pass deterministically in under 3 seconds in CI/CD.

### Q17: What are the next three features you would build given another sprint? [Product Roadmap]

Strong Candidate Response:
1. Hybrid Retrieval (BM25 + Dense Vectors) with Reciprocal Rank Fusion (RRF) to capture rare acronyms and exact code symbols.
2. Cross-Encoder Reranker (e.g., Cohere Rerank or BGE-Reranker-Large) to reorder top-20 retrieved candidates down to top-4.
3. Self-Correction / Corrective RAG (CRAG): If the evaluation engine outputs a FAIL verdict (score < 0.70), automatically rewrite the query and re-retrieve context before delivering the answer to the user.

## 7. QUICK-FIRE REVISION CHEAT SHEET

| | |
|---|---|
| Hallucination (Extrinsic) | |
| Context Noise | Retrieved chunks that match keywords but lack answer signal. Caught by ContextRelevanceEvaluator. |
| LangGraph StateGraph | Cyclic, stateful computational graph framework used to manage RAG execution stages. |
| Pydantic v2 Coercion | Pre-validation logic ensuring malformed vector metadata does not trigger 500 server crashes. |
| Micro-Batching | Grouping chunks into 25-item slices to obey Google Gemini free tier 100 req/min quotas. |
| LLM-as-a-Judge | Using a foundation model to score model outputs against strict multi-criteria rubrics. |
| Cosine Distance | Distance metric 1 - (A.B)/(\|\|A\|\|\|\|B\|\|) used by ChromaDB for nearest neighbor search. |

[FINAL INTERVIEW ADVICE & MINDSET]
1. Lead with practical numbers: '30 live queries, 90.5% quality score, 5-pillar evaluation'.
2. Emphasize why evaluation matters: 'Generative AI without evaluation is a liability in enterprise production.'
3. Speak to tradeoffs: 'Why chunk size 1000? Why LangGraph over LCEL? Why ChromaDB over Pinecone?'
4. Highlight resilience: Talk passionately about the 429 quota fix and Pydantic validation coercion.